UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS, MATEMATICAS Y FISICAS
CARRERA DE SOFTWARE

MATERIA:

GESTION DE LA CONFIGURACION DEL SOFTWARE

TEMA DE ACTIVIDAD:

DOCUMENTO 2 — SDD (IEEE 1016):
DESCRIPCIÓN DEL DISEÑO DEL SOFTWARE

DOCENTE:

ING. ALVAREZ SOLIS FRANCISCO XAVIER

ESTUDIANTE:

HUACON RAMIREZ GREGORY JOSUE

MALDONADO LOPEZ SAUL ALEJANDRO

ROMERO DIAZ ANDY ADRIAN

SOLIS SALAZAR ALEJANDRO SEBASTIAN

TENESACA PINCAY CINTHIA MARIA

CICLO ESCOLAR:

2025 – 2026 CICLO II

Contenido

Contenido

1. Introducción **2**

2. Diseño Arquitectónico Global **2**

2.2. Flujo de Control e Interacciones 3

3. Descripción del Desglose en Módulos Funcionales **4**

4. Diseño de Estructuras de Datos e Interfaces **4**

4.1. Diseño de Interfaces (API REST) 4

4.2. Diseño Lógico de la Base de Datos (Estructuras de Datos) 5

5. Estrategia de Configuración, Pruebas y Despliegue **6**

5.1. Configuración del Entorno de Desarrollo 6

5.2. Estrategia de Control de Versiones 6

5.3. Estrategia de Pruebas 6

5.4. Estrategia de Despliegue (CI/CD) 6

1. Introducción

Define el propósito y el alcance del documento.

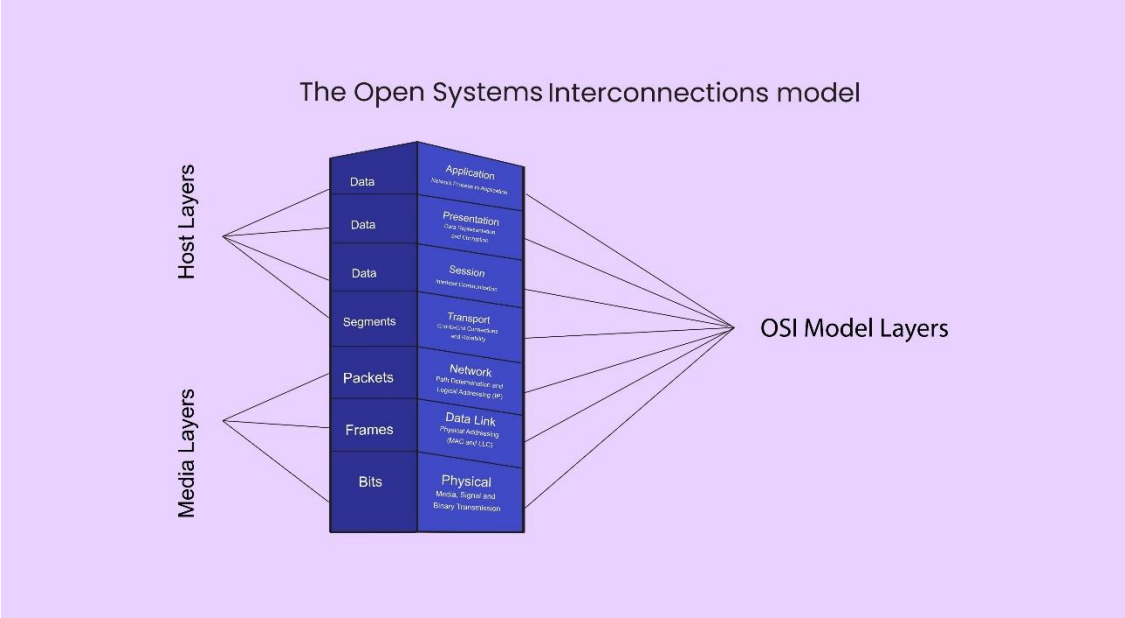
- Propósito: Describir la arquitectura, el diseño de alto nivel y el diseño detallado de los componentes del Sistema de Gestión y Seguimiento para el Taller de Celulares.
- Alcance: Este documento cubre el diseño lógico, la arquitectura, los modelos de datos y la configuración técnica del entorno, desde el Frontend (React) hasta la Base de Datos (MySQL).

2. Diseño Arquitectónico Global

Esta es la sección más importante para definir cómo se construirá el sistema.

2.1. Estilo y Arquitectura de Diseño

El diseño se basará en la arquitectura de Programación por Capas (Layered Architecture), según el requerimiento del cliente.



Capas	Responsabilidad	Tecnología principal
Capa 1: Presentación (Frontend)	Interacción con el usuario (UI/UX) y gestión del estado local.	React JS
Capa 2: Aplicación (Backend/API)	Lógica de negocio, autenticación, autorización y orquestación.	Node.js (Express/NestJS)
Capa 3: Datos (Persistencia)	Almacenamiento, recuperación y gestión transaccional de la información.	MySQL

2.2. Flujo de Control e Interacciones

Se describe cómo fluyen los datos y las interacciones entre las capas:

1. El Frontend (React) realiza una solicitud (ej. GET /devices/status) a la API.
2. El Backend (Node.js) recibe la solicitud. El módulo de Autenticación valida el token (JWT).
3. El Servicio de Negocio (ej. *Device Service*) ejecuta la lógica requerida.
4. El Repositorio de Datos accede a la Base de Datos (MySQL) para obtener o modificar la información.
5. Los datos son devueltos a través de la API al Frontend para su visualización.

3. Descripción del Desglose en Módulos Funcionales

Se desglosa el sistema en componentes (módulos) y se traza su relación con los requerimientos funcionales del SRS.

Módulo Principal	Componentes Lógicos (Backend)	Funcionalidad Clave (RF)
Gestión de Acceso	**Servicio de Autenticación, Repositorio de Usuarios**	**Login, Registro por Rol (Admin, Técnico), Seguridad (JWT).**
Gestión de Clientes	**Servicio de Clientes, Repositorio de Clientes**	**CRUD de Clientes.**
Módulo de Taller/Servicio	**Servicio de Dispositivos, Servicio de Diagnóstico**	**Registro de equipo, Actualización del estado (flujo: Ingresado -> Reparado), Seguimiento.**
Módulo de Inventario	**Servicio de Inventario, Lógica de Stock**	**CRUD de Repuestos, Gestión de alertas de stock crítico.**
Facturación y Reportes	**Servicio de Facturación, Generador de Reportes**	**Emisión de Factura Electrónica, Generación de reportes (Ingresos, Servicios).**
Portal Cliente	**Controlador de Acceso Público, Vistas de Estado**	**Consulta de estado en tiempo real (por ID de Orden).**

4. Diseño de Estructuras de Datos e Interfaces

4.1. Diseño de Interfaces (API REST)

- Protocolo: REST (Representational State Transfer).
- Formato de Intercambio: JSON.
- Endpoints de Ejemplo:
    - POST /api/v1/auth/login

- GET /api/v1/devices/{id} (Consulta estado de un equipo)
- POST /api/v1/inventory (Registro de nuevo repuesto)
- PUT /api/v1/devices/{id}/status (Actualización de estado por el técnico)

- Documentación de API: Se utilizará Swagger/OpenAPI para documentar el contrato de la API y mitigar el riesgo de inconsistencia de datos (como se indica en el SPMP).
- Seguridad de API: Uso obligatorio de JSON Web Tokens (JWT) en todas las rutas privadas.

4.2. Diseño Lógico de la Base de Datos (Estructuras de Datos)

Se definen las entidades principales (tablas) del modelo relacional MySQL que soportarán la persistencia:

Entidad	Propósito	Relaciones Clave
Usuarios	**Gestión de acceso y roles (Admin, Técnico).**	**Relación 1:N con Dispositivo (Técnico asignado).**
Clientes	**Almacena información de contacto y CI.**	**Relación 1:N con Dispositivo y Factura.**
Dispositivos	**Almacena datos del equipo, estado actual y número de serie.**	**Relación 1:1 con Diagnóstico (en curso) y Factura.**
Inventario	**Almacena datos de repuestos y su stock.**	**Relación N:M con Diagnóstico/Reparación.**
Diagnóstico	**Detalla fallas, repuestos usados y costo.**	**Relación 1:1 con Dispositivo.**
Facturas	**Documento de cobro (incluye total, impuestos).**	**Relación N:1 con Cliente.**

5. Estrategia de Configuración, Pruebas y Despliegue

5.1. Configuración del Entorno de Desarrollo

- Lenguajes: JavaScript/TypeScript.
- Herramientas: Git/GitHub o GitLab (para control de código), Jira o Trello (para gestión de tareas).
- Estándares de Código: Uso de ESLint y Prettier para mantener uniformidad y calidad del código (según SPMP).

5.2. Estrategia de Control de Versiones

Se implementará el flujo de trabajo Gitflow para garantizar la estabilidad del código y las ramas de producción y desarrollo.

- Ramas Clave: main (Producción), develop (Integración), feature/* (Desarrollo de funcionalidades).
- Política: Todo cambio debe pasar por un Pull Request y una Revisión de Código (Code Review) obligatoria por parte del Líder Técnico o QA Tester antes de su fusión.

5.3. Estrategia de Pruebas

Se emplearán múltiples niveles de pruebas para alcanzar el Plan de Calidad definido:

- Pruebas Unitarias: Aplicadas con Jest, Vitest o Mocha a la lógica de negocio crítica (ej. cálculo de facturas, lógica de stock).
- Pruebas de Integración: Ejecutadas para validar la comunicación entre el Frontend y los *endpoints* del Backend.
- Pruebas de Sistema/Funcionales: Orientadas a validar los Casos de Uso del SRS (CU-01 a CU-14).

5.4. Estrategia de Despliegue (CI/CD)

- Metodología: Implementación de Integración Continua / Despliegue Continuo (CI/CD).
- Proceso: El *pipeline* automatizado debe ejecutar las pruebas unitarias y de integración antes de permitir el despliegue a los entornos de *staging* y Producción.
- Entorno Final: El sistema se desplegará en un entorno Cloud (p. ej., AWS, Vercel, Render) para garantizar la disponibilidad 24/7 (RNF-004)